Securing UAV Command and Control Against Quantum Adversaries: Hybrid, KEM Authenticated and Signature Based Post Quantum MAVLink Designs Compared on RISC-V Cores and ArduPilot

Huda Mueen
Dept. of Data Science & AI
University of Stirling
Ras Al Khaimah, United Arab Emirates
hudamueen2407@gmail.com

Awaiz Ahmed
Dept. of Software Engineering
Al Ain University
Abu Dhabi, United Arab Emirates
awaiz42ad@gmail.com

***Abstract*—Drone flight controllers fielded today may still fly when an attacker can break classical public key cryptography. This measurement study compares seven MAVLink command channel designs under one set of conditions: MAVLink 2 signing, classical (X25519, Ed25519), post quantum only (ML-KEM-768, ML-DSA-44) and hybrid sessions, a session authenticated by pinned static ML-KEM-768 keys (design K), and an ML-DSA-44 or a Falcon-512 signature on every command. In cycle accurate simulation of two open RISC-V cores, the hybrid session costs 1.75 to 1.89 times the vehicle side computation of the classical session and design K 1.03 to 1.13 times. Gateways that execute the cryptography in front of ArduPilot in software in the loop, over an emulated telemetry radio, show that a session keeps the per command cost of the shipped tag and misses almost no more commands than no authentication. With a signature per command, 0.418 of commands meet a 0.5 s deadline with ML-DSA-44 and 0.913 with Falcon-512, whose signing costs 9.7 times as many cycles with emulated floating point. Over 100 missions per configuration, a strict counter rule misses +1.48 commands per mission more than the published rule after outages, and still +1.04 when it resynchronises inside the session; a replay cache misses almost none more (+0.05). In ProVerif, a replay cache gives injective agreement on commands only if each counter is checked and recorded in one step, and design K loses key secrecy once ML-KEM alone is broken. The code, data, logs and scripts are public at https://github.com/hudasol/post-quantum-security.**

I. Introduction

A small multirotor runs its flight stack on a microcontroller and receives commands over a telemetry radio of tens of kilobits per second. MAVLink 2, the common protocol, offers a per message tag with a pre shared key [1]; without it, commands can be injected from the radio side [2], [3], and key establishment and endpoint authentication are left to the operator. A pre shared key is not broken by a quantum computer, but one stolen key exposes every session that used it, and a fleet must provision and rotate such keys by hand. A session key exchange removes both problems, but its public key part is what a quantum computer threatens during the years a vehicle stays in service. The replacements are standardised: ML-KEM for key establishment [4] and ML-DSA for signatures [5].

This is a measurement study, and its building blocks are known: design K follows KEMTLS with pre distributed keys and post quantum WireGuard [6], [7], the hybrid key derivation follows hybrid KEM combiners such as X-Wing [8], and the replay cache is the sliding receive window of IPsec [9]. Hyoung et al. argue from post quantum TLS handshakes over an emulated drone radio that, after the handshake, commands should carry only a symmetric tag, but derive rather than measure the per command cost [10]. Missing is a comparison of complete designs on the same core, link and flight stack, including what the replay rule does to commands.

This paper contributes: (1) executed baselines of seven command channel designs on two open RISC-V core designs in cycle accurate register transfer level (RTL) simulation, with correctness checks, intervals over 15 runs, stack and code size, under one compiler and pinned libraries (Section V); (2) a gateway emulation in front of ArduPilot software in the loop (SITL) that executes the handshakes, tags, signatures and replay rules on paired seeds, also under burst loss and over 100 missions per configuration (Section VI); (3) ProVerif models of the hybrid and the KEM authenticated sessions and of replay rules that accept out of order delivery, with the bounds of each model (Section VII). Every number in this paper is read by script from a committed result file.

II. Related work

Primitives are benchmarked on Cortex-M4 with pqm4 [11], with stack optimised Dilithium [12] and with RISC-V extensions [13], [14]. Post quantum TLS has been measured on embedded clients [15] and under packet loss [16], [17]; KEMTLS [6], [18], post quantum WireGuard and post quantum Noise [7], [19] authenticate by encapsulation instead of signatures. For drones, MAVSec encrypts MAVLink [20], Ahmed et al. combine two ML-KEM-768 encapsulations with ML-DSA-65 certificates and compute radio times without a flight stack [21], and de Meneses et al. verify a post quantum EDHOC variant with estimated costs [22]; for vehicle to vehicle messages,

frame loss can decide practicality [23]. Table I compares the closest drone and post quantum protocol studies by method, not figures, because platforms and links differ.

III. Threat model and designs

Threat model. The attacker controls the radio: it reads, drops, delays, reorders and injects frames and replays old ones, and it may hold a quantum computer during the service life of the vehicle. Endpoints, their long term keys and the flight stack are trusted; side channels, jamming beyond the modelled loss and outages, and the confidentiality of commands are out of scope. So is resource exhaustion: an injected first flight makes the vehicle compute for 103 ms (design K) or 66 ms (hybrid) at the applied clock before anything is authenticated, so a deployment needs rate limiting. The goals are mutual authentication of the session endpoints, secrecy of the session key that keys the tags, and injective agreement on commands: every accepted command was sent by the ground station in that session and is accepted at most once. Per message protection stays the MAVLink 2 tag, SHA-256 over key, frame and counter truncated to 48 bits [1], so a forged frame succeeds with probability 2^{-48}, and every attempt must be sent over the radio.

Session designs. All sessions end with the MAVLink 2 signing field of 13 B per message, keyed by the session key. *Classical*: X25519 key exchange, Ed25519 signatures by both sides over the transcript and both identities, key confirmation tags. *Post quantum only*: ML-KEM-768 and ML-DSA-44 in the same roles. *Hybrid* (proposed): X25519 and ML-KEM-768 together, the session key derived from both secrets and a transcript hash, ML-DSA-44 signatures by both sides; a variant in which only the ground station signs leaves the vehicle unauthenticated in our model, so the vehicle signs too (mutual signing). *KEM authenticated* (design K): each side holds the other's static ML-KEM-768 key; the ground station sends an X25519 share, an ephemeral ML-KEM-768 key and an encapsulation to the vehicle's static key, the vehicle answers with its share, a ciphertext to the ephemeral key, an encapsulation to the ground station's static key and a confirmation tag, and the ground station confirms; there are no signatures. Fig. 1 shows the flights of the hybrid session and of design K with their payload bytes; the classical and post quantum only sessions carry 224 B and 7,144 B in four flights, without MAVLink framing.

Baselines without a session. Signing as shipped: a provisioned key, no handshake. *Signature per command*: an ML-DSA-44 or a Falcon-512 signature on every command, verified by the vehicle; as in earlier per command designs it carries no counter, so it does not resist replay and is weaker than the sessions; a second variant of each adds a 6 B counter under the published rule, so it resists replay.

Replay rules. MAVLink 2 signing carries a 48 bit timestamp per stream, and its *published* rule accepts a frame whose timestamp exceeds the last one accepted on that stream [1]; our gateways put a per session counter into that field and apply the same rule, so gaps are allowed. The *strict* rule accepts only the next counter; when the ground station's link layer gives up a frame, or when the vehicle sees a later counter and the next one does not arrive, the two sides resynchronise, either by a full new handshake or, in the second variant, by one authenticated message each way under the existing session key. The *replay cache* accepts any counter not yet accepted within a window of 64 counters, the default receive window of IPsec [9].

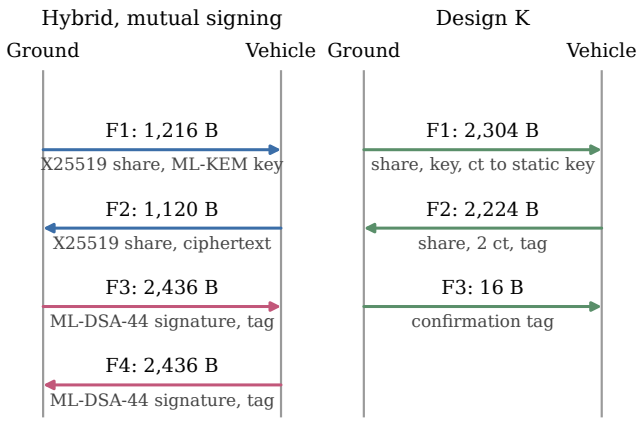


Fig. 1. Handshake flights of the hybrid session with mutual signing (left) and of design K (right), with their bytes as built; pink arrows carry ML-DSA-44 signatures.

IV. Method

RTL simulation. Ibex in its small configuration [24] and CV32E40P [25], both RV32IMC without caches or wait states, are simulated cycle accurately with Verilator. One firmware image per design runs the vehicle and the ground side, each step timed with the cycle counter, 15 runs per session image with a fixed seed block (5 for the deterministic tag image); every run checks that both sides derive the same key, that all signatures and tags verify, and hash known answers; medians carry 95% intervals over runs. These are bare metal cycles with single cycle memory, not board cycles. Libraries are pinned and unmodified: mlkem-native [26], mldsa-native [27], Monocypher [28] and the padded Falcon-512 of liboqs [29], [30], all portable C, compiled with one compiler at `-O3`, so that classical and post quantum code stand on the same footing, without assembly; neither core has a floating point unit, so Falcon runs its integer emulation of floating point. Peak stack is measured by stack painting and includes the harness; code size is the text section of each image with unused sections removed, both sides and the harness included.

Gateway emulation with ArduPilot SITL. ArduPilot Copter 4.7.2 [31] flies a scripted mission of about 48 s (take off, two legs, land) with its built in vehicle model; the ground station sends one acknowledged COMMAND_LONG message per second. A relay emulates a SiK class telemetry radio of 64 kbit/s with independent frame loss of 0.02, link layer retransmission that can deliver frames out of order, and one outage per mission starting 15–40 s after arming with an

TABLE I
CLOSEST PRIOR WORK AND THIS PAPER (AS RECORDED FROM EACH SOURCE)

Work	Platform	Link and stack	Per command cost	Availability	Formal check
[10]	x86 laptop	emulated UAV radio, TLS 1.3	derived, not measured	not reported	none reported
[7]	virtual machines	virtual network, WireGuard	not reported	not reported	Tamarin, with replay
[21]	x86 laptop	none; SiK radio times computed	not reported	not reported	proof on paper
[22]	modelled Cortex-M4	UAV, EDHOC variant, computed delays	not reported	not reported	Tamarin
This paper	two RISC-V cores (RTL)	emulated SiK radio, ArduPilot SITL	executed (RTL)	measured per rule	ProVerif

exponential duration of mean 5 s; an outage longer than 10 s tears the session down. A gateway pair between the ground station, the relay and the flight stack executes the handshake, the tags and the replay rule with the same algorithms (Python cryptography and host builds of the same libraries); the flight stack itself runs unmodified and never executes the cryptography. Vehicle side operations are released after a delay equal to their Ibex cycles at 168 MHz, the clock of a Cortex-M4 flight controller, fixed from the first runs before the flights (for design K, flown later, from the medians of all runs); ground side compute runs at host speed. The configurations fly 20 missions each on the same seeds, so they see the same outages; mission lengths differ slightly, so the number of commands per configuration ranges from 958 to 972. We therefore compare configurations by the commands missed per mission, paired by mission, with bootstrap intervals over missions. The host was slower than the applied delay for 3,979 of 254,813 vehicle side tags (at most 1,284 in one configuration); these are counted, not corrected. Two further sets use the same seeds: under a two state burst loss model with the same mean frame loss and a mean burst of 5 frames (an assumption without a measured radio source), 20 missions each, and under independent loss 100 missions each, for nine configurations; all ran in two lanes at once, and the Falcon-512 configurations were added after the first runs.

Formal models. ProVerif [32] checks secrecy, agreement and reachability under a Dolev Yao attacker with perfect abstract primitives and unbounded sessions; binding properties of the KEMs are not analysed. Session models carry one command, a rekey and a further command per session; replay cache models carry three commands per key; the bounded window is modelled on the command layer for a window of two counters, not the 64 flown. Rekey grace periods, counter wrap and timestamps are not modelled. Breaking a component is modelled by giving its secrets to the attacker.

Discipline. Of 12 technical predictions hashed before the runs and scored by script under the less favourable reading, 10 hit, 1 fell between and 1 missed; the thresholds were broad, and design K was added after registration. A second set of 17 predictions for the Falcon-512, burst loss and larger runs was registered later, after design K was added, and scored the same way: 16 hit, 1 fell between and 0 missed.

V. EXECUTED BASELINES ON CORE DESIGNS

Table II gives the cost of one session start and of one command. On CV32E40P the hybrid session costs 1.75× [1.59, 1.77] the classical vehicle side, and on Ibex 1.89× [1.72, 1.91]. Against the post quantum only session the hybrid costs 1.26× [1.05, 1.41] on the vehicle side and 1.55× on the ground side of CV32E40P. The two designs differ by two X25519 operations, a fixed 6.32 M cycles, yet their measured medians differ by only 4.48 M: ML-DSA-44 signing repeats with rejection and took 5.6 to 39.2 M cycles per signature, so the medians of the signing designs are noisy, and the sums of primitive medians drawn in Fig. 2 exceed the measured session medians. With Monocypher, one X25519 multiplication (3.16 M cycles on CV32E40P) costs more than an ML-KEM-768 encapsulation (1.70 M), and an Ed25519 verification 4.53 M; this is a property of one portable C library designed for size, and a faster classical library would raise every ratio over the classical session.

TABLE II
EXECUTED BASELINES: MILLION CYCLES PER UNIT (RTL SIMULATED, MEDIANS OF 15 RUNS, 5 FOR THE TAG ROW) AND CODE SIZE ON CV32E40P (HOST MEASURED)

	Vehicle		Ground		Code
Design	Ibex	CV32E40P	Ibex	CV32E40P	(kB)
MAVLink 2 tag[a]	0.015	0.013	0.014	0.013	21.7
Classical session	15.5	12.4	15.4	12.4	44.9
PQ only session	23.8	17.3	21.6	15.4	56.1
Hybrid session	29.2	21.8	32.3	23.9	66.5
KEM auth. session (K)	17.4	12.8	20.2	14.7	46.0
ML-DSA-44 per command[b]	5.4	3.9	10.0	7.4	37.8
Falcon-512 per command[b,c]	1.15	0.91	77.9	72.4	98.7

Session rows: one session start. [a] check (vehicle) and generation (ground) of one tag over a COMMAND_LONG frame. [b] verification (vehicle) and signing (ground) of one command. [c] padded Falcon-512 with integer emulated floating point; key pair made once.

Design K avoids both ML-DSA-44 operations: on CV32E40P its vehicle side costs 1.03× the classical session and 0.59× [0.55, 0.65] the hybrid one, it needs 18.8 kB of stack against 50.5 kB for the hybrid (2.1 kB classical), and it carries 4,544 B in three flights against 7,208 B for the hybrid session, which carries 32× the bytes of the classical handshake. Code for both sides and the harness grows from 44.9 kB (classical) to 66.5 kB (hybrid) and 46.0 kB (design K). The 5 ArduPilot builds we measured leave 102 to 377 kB of flash and at least 111 kB of static RAM free (host measured from linker output), so the code fits, but a 50.5 kB stack would need a thread of its own or a stack optimised signer [12]; a vehicle only image was not measured. After the handshake a command costs a tag check of 13,314 cycles

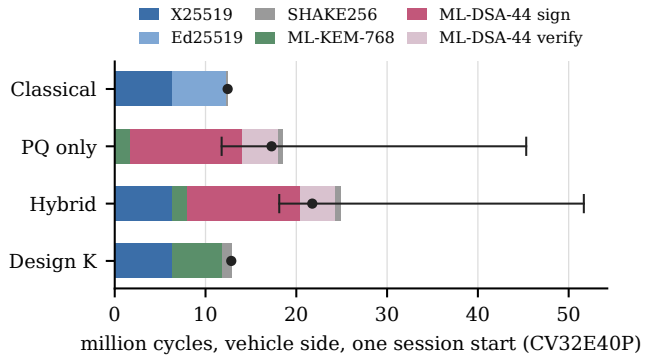


Fig. 2. Vehicle side cycles of one session start on CV32E40P by primitive (RTL simulated). Bars are sums of primitive medians, which exceed the measured medians of the signing designs; the whisker is the bar total if its one signature took the fastest or the slowest ML-DSA-44 signing observed; the dot is the measured session median.

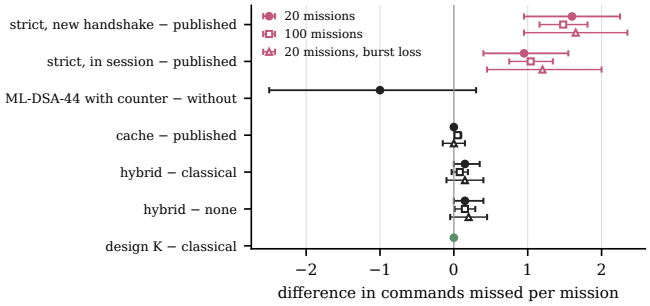


Fig. 3. Paired differences between configurations of the commands missed per mission (not acknowledged within 0.5 s, outages included), mean over missions with 95% bootstrap interval (SITL measured). Filled circles: the 20 missions of Table III; open squares: 100 missions; open triangles: 20 missions under burst loss.

on a COMMAND_LONG (the tag alone, without parsing); verifying an ML-DSA-44 signature on the same frame costs 292× as much on CV32E40P, and signing it on the ground side about twice that. A Falcon-512 signature (666 B) is verified in 0.91 M cycles on CV32E40P, 0.23× the ML-DSA-44 verification, with 5.0 kB of stack, but signing with emulated floating point takes 72.4 M cycles, 9.7× ML-DSA-44 signing; on an emulated Cortex-M4 (QEMU, functional only, no cycle counts) the same code signs, verifies and rejects altered input at all 3 optimisation levels tried. Ibex needs 1.34× the cycles of CV32E40P for the same handshake, so cycle counts do not transfer between cores.

VI. Gateway emulation with ArduPilot SITL

Sessions. Table III gives each configuration against no authentication; missed commands per mission is the common footing for comparing rows. Once a session exists, its per message cost is that of the shipped tag: the hybrid session adds 4.4 ms to the median command round trip, the classical session 4.3 ms and signing as shipped, which runs inside the flight stack without gateways, 4.0 ms. These milliseconds are mostly airtime of the 13 B signing field (1.6 ms per direction). The hybrid handshake takes 1.49 s over the emulated radio against 0.19 s for the classical session, a cost paid once per session, and design K completes in 0.92 s. Signing as shipped missed 17 commands that arrived after a newer frame.

Signature per command. An ML-DSA-44 signature on every command adds 496.1 ms to each round trip, mostly airtime of the 2.4 kB signature, since the applied verification delay is 32 ms; this puts the median round trip next to the deadline: 0.418 of its commands are acknowledged within 0.5 s and 0.851 within 1 s.

Replay rules. Fig. 3 compares configurations by the commands missed per mission; right of zero, the first of a pair misses more, and open markers repeat the pair over more missions and under burst loss. Most session configurations differ in at most three of the 20 missions: the hybrid session misses +0.15 commands per mission against no authentication and +0.15 against the classical session, a small cost from re-handshakes after long outages; design K and the classical session miss the same commands (+0.00). Link layer retransmission delivers some frames after newer ones (reordering depth up to 2), and the published rule rejected 11 of them as late; the replay cache accepted them, but they had already missed the deadline, so the cache misses exactly as many commands as the published rule (+0.00, 0 missions differ).

The strict rule is the one that costs commands. Resynchronising by a full new handshake, it misses +1.60 commands per mission against the published rule (+0.95 to +2.25), mainly through resynchronisation, not reordering: 58 commands were lost while no session existed during 12 resynchronisations, 10 of them triggered when the link layer gave up a frame during an outage, against 13 rejected as ahead of the expected counter. Resynchronising inside the session removes the handshake but not the cost: across 22 resynchronisations it still misses +0.95 commands per mission against the published rule (+0.40 to +1.55), -0.65 against the strict rule with a new handshake, because every command sent while the counter is out of step is rejected. Adding a counter to the signature per command does not change its availability (-1.00 commands per mission, -2.50 to +0.30).

Falcon-512. A Falcon-512 signature per command, with 6.8 ms of verification at the applied clock, adds 124.5 ms to the round trip, mostly the airtime of its 666 B signature, and 0.913 [0.874, 0.949] of its commands are acknowledged within 0.5 s, against 0.418 for ML-DSA-44. Its missed commands per mission differ by -24.05 from ML-DSA-44 (-26.25 to -22.00) and by +0.90 from the hybrid session, and its counter changes nothing (+0.00). The price is ground side signing: 6.4 ms on the host and many times ML-DSA-44 on the cores (Section V).

Burst loss and more missions. The replay rule results hold under burst loss and over more missions (Fig. 3). Under burst loss the strict rule with a new handshake misses +1.65 commands per mission against the published rule (+0.95 to +2.35), in session +1.20, and the cache +0.00. Bursts did not hurt per command signatures more than sessions: from independent to burst loss, missed commands per mission

TABLE III
ARDUPILOT SITL BEHIND THE GATEWAY EMULATION AND AN EMULATED SIK CLASS RADIO, 20 PAIRED MISSIONS PER CONFIGURATION

Configuration	Handshake (s)	Added round trip (ms)	Share acknowledged within 0.5 s [95% CI]	Missed per mission
No authentication	–	ref.	0.935 [0.902, 0.963]	3.15
MAVLink 2 signing as shipped[a]	–	4.0	0.932 [0.902, 0.962]	3.30
Classical session	0.19	4.3	0.935 [0.904, 0.965]	3.15
Hybrid, published rule	1.49	4.4	0.931 [0.896, 0.961]	3.30
Hybrid, strict rule, new handshake	1.50	4.3	0.898 [0.858, 0.936]	4.90
Hybrid, strict rule, in session	1.49	4.8	0.912 [0.872, 0.947]	4.25
Hybrid, replay cache	1.44	4.2	0.932 [0.899, 0.964]	3.30
Design K, published rule	0.92	4.7	0.934 [0.903, 0.964]	3.15
ML-DSA-44 per command	–	496.1	0.418 [0.374, 0.459]	28.25
ML-DSA-44 per command, counter	–	493.8	0.439 [0.403, 0.477]	27.25
Falcon-512 per command[b]	–	124.5	0.913 [0.874, 0.949]	4.20
Falcon-512 per command, counter[b]	–	127.5	0.913 [0.874, 0.948]	4.20

Handshake: median from F1 until both sides hold the key (vehicle accepting F3 for design K, ground accepting F4 otherwise). Added round trip: median round trip to the flight stack's acknowledgement against no authentication (SITL measured). Acknowledged share and missed commands (not acknowledged within 0.5 s) include one outage per mission. [a] signing inside ArduPilot and the ground station at host speed, without RTL derived delays. [b] flown after the other configurations, on the same seeds.

changed by -1.85 for ML-DSA-44 per command and +0.10 for the hybrid session, probably because bursts hit fewer of the many frames of one signed command (not checked frame by frame). Over 100 missions under independent loss, the strict rule misses +1.48 (+1.16 to +1.81) and in session +1.04 (+0.75 to +1.34) against the published rule, against +1.60 and +0.95 over the first 20; the cache misses +0.05 more (+0.01 to +0.10), so it is not exactly even with the published rule. Reruns of the first 20 seeds moved configuration means by -0.35 to 0.30, as the flight stack runs in real time.

VII. FORMAL RESULTS

Scope. Results are symbolic (Dolev Yao, perfect primitives), not computational proofs. ProVerif checked 240 queries in 39 model files: 102 proved, 126 counterexamples, 3 that the tool could not prove and 9 that reached the time limit. Of the counterexamples, 71 are reachability checks that must fail, so the agreement results are not vacuous; the others come from designs before their fixes, other models, controls and the results below. With mutual signing each side of the hybrid session authenticates the other, and its keys stay secret, also with either component broken or the long term keys later stolen. With ML-DSA-44 broken (both signing keys known to the attacker), authentication fails and an active attacker learns the key a side accepts; only a session that both sides accepted with the same key stays secret, which the X25519 and ML-KEM exchange still gives.

Design K. Design K keeps its keys secret when the classical component is broken and after a later theft of the static keys, and the vehicle authenticates the ground station. With ML-KEM broken, however, its keys are not secret: the attacker sends its own first flight with stolen static keys, and the X25519 share does not save the session against an active attacker; with its static keys known, design K fares like the hybrid session with ML-DSA-44 broken. Authentication of the vehicle by the ground station is proved only in a model in which static and ephemeral KEM keys have distinct types; the general model reached a time limit of 3 h without finding an attack or a proof; this is a limit of the tool over unbounded sessions, not an attack.

Replay rules. For out of order delivery, a replay cache proves injective agreement on commands when the vehicle checks and records each pair of session and counter in one step and never forgets an entry, a condition stated in the model as a restriction, not derived. When concurrent branches read a table before either records, the model gives a counterexample, and a table read in sequence could not be proved; an implementation must therefore check and record atomically. A bounded window was proved only on the command layer for a window of two counters, and the published timestamp rule could not be proved.

VIII. DISCUSSION AND LIMITATIONS

For streamed commands the results support a session, whose post quantum cost is paid once. Where both identities can be pinned, design K is cheaper but rests on ML-KEM alone; the hybrid session keeps the classical component as a guard against a flaw in ML-KEM, while its authentication rests on ML-DSA-44. An ML-DSA-44 signature per command fits only commands that can wait about a second; a Falcon-512 signature (710 B against 2,464 B per signed command here) keeps most commands on time at a small vehicle cost, if the ground side can afford signing at 9.7 times the cycles of ML-DSA-44 with emulated floating point, or has a floating point unit, which our cores lack.

Threats to validity. (i) Cycles come from two open core designs with single cycle memories, applied at a Cortex-M4 clock; Cortex-M flight controllers have flash wait states, caches and other multipliers, which add unmeasured cycles, most for code that streams large keys and signatures or calls many small routines (ML-DSA-44, Falcon-512 with emulated floating point); an earlier run of portable C code on Ibex took 1.6 to 2.9 times the published portable C cycles on Cortex-M4 and 3.2 to 4.3 times its assembly for ML-KEM-768 and ML-DSA-44; stack optimised signing [12] or RISC-V extensions [13], [14] would change the figures, and the Cortex-M4 check

of Falcon-512 is functional only. (ii) One classical library was measured, and it favours the post quantum designs. (iii) The cryptography runs in gateways, not in the flight stack, so the scheduling load of a handshake on the flight controller is not measured; vehicle compute is a delay applied per event without a busy processor, and the host was slower than that delay for a small share of tags. (iv) The radio is an emulation with one scripted outage per mission; burst loss is covered only by one two state model with one assumed burst length, not by a radio trace, and the strict rule results rest on outage triggered events. (v) Twenty missions per configuration for most results and 100 for nine configurations, a bootstrap over missions that may under cover, and no correction for the several paired comparisons. (vi) The formal models use perfect primitives and small bounds (Section IV). Next, the baselines should be repeated on Cortex-M boards with the vehicle side inside the flight stack and a real radio.

IX. Conclusion

Executed under one set of conditions, a hybrid post quantum session for drone commands costs 1.75 to 1.89 times the vehicle side computation of a classical session on these cores and many times its handshake bytes, yet keeps the per command cost of the shipped tag; a KEM authenticated session with pinned keys is the cheapest post quantum option but rests on ML-KEM alone. An ML-DSA-44 signature per command misses a 0.5 s deadline for most commands, while a Falcon-512 signature keeps most commands on time at 9.7 times the signing cost. The replay rule matters as much as the algorithms: a strict rule loses commands after outages, with or without a new handshake, over more missions and under burst loss, while a replay cache loses almost none against the published rule and is safe only if it checks and records each counter in one step.

References

[1] MAVLink project, "Message Signing (Authentication)," MAVLink Developer Guide, https://mavlink.io/en/guide/message_signing.html, 2026.

[2] Y.-M. Kwon, J. Yu, B.-M. Cho, Y. Eun, and K.-J. Park, "Empirical Analysis of MAVLink Protocol Vulnerability for Attacking Unmanned Aerial Vehicles," *IEEE Access 6, 43203-43212*, 2018, DOI 10.1109/ACCESS.2018.2863237.

[3] A. Koubaa, A. Allouch, M. Alajlan, Y. Javed, A. Belghith, and M. Khalgui, "Micro Air Vehicle Link (MAVLink) in a Nutshell: A Survey," *IEEE Access (accepted June 2019, as stated on the arXiv copy)*, 2019, https://arxiv.org/pdf/1906.10641.

[4] National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard (FIPS 203)," Federal Information Processing Standards, https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf, 2024.

[5] ——, "Module-Lattice-Based Digital Signature Standard (FIPS 204)," Federal Information Processing Standards, https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf, 2024.

[6] P. Schwabe, D. Stebila, and T. Wiggers, "More efficient post-quantum KEMTLS with pre-distributed public keys," in *Proc. ESORICS 2021*, 2021, DOI 10.1007/978-3-030-88418-5_1.

[7] A. Hülsing, K.-C. Ning, P. Schwabe, F. J. Weber, and P. R. Zimmermann, "Post-quantum WireGuard," in *Proc. IEEE S&P 2021*, 2021, Cryptology ePrint Archive 2020/379.

[8] M. Barbosa, D. Connolly, J. D. Duarte, A. Kaiser, P. Schwabe, K. Varner, and B. Westerbaan, "X-Wing: The Hybrid KEM You've Been Looking For," *IACR Communications in Cryptology*, 2024, Cryptology ePrint Archive 2024/039.

[9] S. Kent, "IP Encapsulating Security Payload (ESP)," RFC 4303, IETF, https://datatracker.ietf.org/doc/html/rfc4303, 2005.

[10] Y. Hyoung, D. Lim, J. Bang, D. Park, H. Kim, and H. Seo, "Post-Quantum Command Authentication for Unmanned Systems: A KpqC Feasibility Study under Real-Time and Bandwidth Constraints," Cryptology ePrint Archive 2026/2343, https://eprint.iacr.org/2026/2343.pdf, 2026.

[11] M. J. Kannwischer, J. Rijneveld, P. Schwabe, and K. Stoffelen, "pqm4: Testing and Benchmarking NIST PQC on ARM Cortex-M4," Cryptology ePrint Archive 2019/844, https://eprint.iacr.org/2019/844.pdf, 2019.

[12] D. O. C. Greconici, M. J. Kannwischer, and A. Sprenkels, "Compact Dilithium Implementations on Cortex-M3 and Cortex-M4," *IACR TCHES 2021(1), 1-24*, 2021, DOI 10.46586/tches.v2021.i1.1-24.

[13] E. Alkim, H. Evkan, N. Lahr, R. Niederhagen, and R. Petri, "ISA Extensions for Finite Field Arithmetic: Accelerating Kyber and NewHope on RISC-V," *IACR TCHES 2020(3), 219-242*, 2020, DOI 10.13154/tches.v2020.i3.219-242.

[14] T. Fritzmann, G. Sigl, and J. Sepúlveda, "RISQ-V: Tightly Coupled RISC-V Accelerators for Post-Quantum Cryptography," *IACR TCHES 2020(4), 239-280*, 2020, DOI 10.13154/tches.v2020.i4.239-280.

[15] K. Bürstinghaus-Steinbach, C. Krauß, R. Niederhagen, and M. Schneider, "Post-Quantum TLS on Embedded Systems," in *Proc. AsiaCCS 2020*, 2020, Cryptology ePrint Archive 2020/308.

[16] C. Paquin, D. Stebila, and G. Tamvada, "Benchmarking Post-Quantum Cryptography in TLS," in *Proc. PQCrypto 2020*, 2020, Cryptology ePrint Archive 2019/1447.

[17] D. Sikeridis, P. Kampanakis, and M. Devetsikiotis, "Post-Quantum Authentication in TLS 1.3: A Performance Study," in *Proc. NDSS 2020*, 2020, DOI 10.14722/ndss.2020.24203.

[18] P. Schwabe, D. Stebila, and T. Wiggers, "Post-Quantum TLS without Handshake Signatures," in *Proc. ACM CCS 2020*, 2020, Cryptology ePrint Archive 2020/534.

[19] Y. Angel, B. Dowling, A. Hülsing, P. Schwabe, and F. J. Weber, "Post Quantum Noise," in *Proc. ACM CCS 2022*, 2022, DOI 10.1145/3548606.3560577.

[20] A. Allouch, O. Cheikhrouhou, A. Koubâa, M. Khalgui, and T. Abbes, "MAVSec: Securing the MAVLink Protocol for Ardupilot/PX4 Unmanned Aerial Systems," in *Proc. IWCMC 2019*, 2019, https://arxiv.org/pdf/1905.00265.

[21] S. Ahmed, M. H. Anisi, A. Iqbal, A. Suresh, M. Amoon, and K. Agarwal, "Quantum Safe Drone Security: Authenticated Key Exchange and Privacy Preserving Intrusion Detection for Low Altitude Intelligent Transportation Systems," *Computers and Electrical Engineering 139, 111429*, 2026, https://doi.org/10.1016/j.compeleceng.2026.111429.

[22] R. D. de Meneses, C. Teixeira, and M. A. Henriques, "Design, Formal Verification, and Evaluation of a Post-Quantum EDHOC Variant for UAV Communications," in *Proc. SBSeg 2026, pages 472-487*, 2026, DOI 10.5753/sbseg.2026.27127.

[23] G. Twardokus, N. Bindel, H. Rahbari, and S. McCarthy, "When Cryptography Needs a Hand: Practical Post-Quantum Authentication for V2V Communications," in *Proc. NDSS 2024*, 2024, https://par.nsf.gov/servlets/purl/10492326.

[24] lowRISC, "Ibex RISC-V Core," GitHub repository, commit 1b29dae8 (v2.1.0-242-g1b29dae8), https://github.com/lowRISC/ibex, 2026.

[25] OpenHW Group, "CV32E40P RISC-V Core," GitHub repository, tag cv32e40p_v1.8.3, https://github.com/openhwgroup/cv32e40p, 2026.

[26] PQ Code Package, "mlkem-native," GitHub repository, tag v2.0.0, https://github.com/pq-code-package/mlkem-native, 2026.

[27] ——, "mldsa-native," GitHub repository, tag v2.0.0, https://github.com/pq-code-package/mldsa-native, 2026.

[28] Monocypher project, "Monocypher," GitHub repository, tag 4.0.3, https://github.com/LoupVaillant/Monocypher, 2026.

[29] O. Q. Safe, "liboqs," GitHub repository, tag 0.16.0, https://github.com/open-quantum-safe/liboqs, 2026.

[30] P.-A. Fouque, J. Hoffstein, P. Kirchner, V. Lyubashevsky, T. Pornin, T. Prest, T. Ricosset, G. Seiler, W. Whyte, and Z. Zhang, "Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU," project page, https://falcon-sign.info/, 2026.

[31] ArduPilot project, "ArduPilot," GitHub repository, tag Copter-4.7.2, https://github.com/ArduPilot/ardupilot, 2026.

[32] B. Blanchet, B. Smyth, V. Cheval, and M. Sylvestre, "ProVerif 2.05: Automatic Cryptographic Protocol Verifier: User Manual and Tutorial," Inria, https://bblanche.gitlabpages.inria.fr/proverif/manual.pdf, 2023.